Applied Programming

Week 10: Priority Queues and Cost-Aware Search

Choosing a low-cost route and finding intermediate robot postures

School of Mechanical Engineering, Soongsil University

Week 9 found routes through a supplied graph of robot postures. This week asks two questions: when movements have different costs, which route should we choose? When useful intermediate postures are missing, how can we find them?

Dijkstra and A* answer the first question using path costs. RRT grows new connections while searching. Both can produce an ordered sequence of robot configurations for the supplied runner.

1 Fewest edges does not mean lowest cost

A **weighted graph** assigns a numerical cost to each edge. The **path cost** is the sum along a route. In Figure 1, taking fewer edges costs more. The printed weights are teaching-example cost units, not drawing distances.

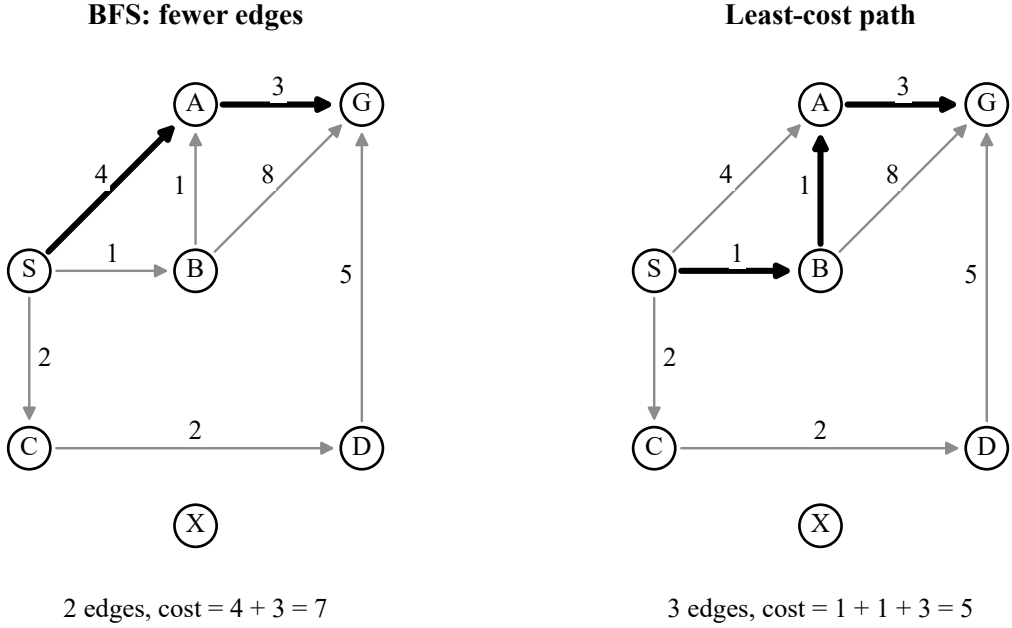


Figure 1: BFS finds two edges with cost 7. The least-cost route uses three edges with cost 5.

The complete graph is supplied in `src/ap_week10/cases.py`. An entry such as `GRAPH["S"] = (("A", 4.0), ("B", 1.0), ("C", 2.0))` lists neighbors and edge costs, not coordinates. X is isolated. All costs are finite and nonnegative.

2 Priority queues and Dijkstra's algorithm

2.1 Remove the smallest priority first

Week 9 used `deque.append()` and `deque.popleft()` for FIFO: remove the earliest arrival. A deque supports both ends, but these calls do not select the lowest cost. A **priority queue** instead removes the smallest numerical priority first. For arrivals A(4), B(1), C(2), FIFO selects A, whereas a priority queue selects B.

Python's standard-library `heapq` supplies this behavior using a **min-heap**. No additional package is needed. `heappush(waiting, item)` adds an item. `heappop(waiting)` removes and returns the smallest one. The list is not fully sorted.

The complete `examples/heap_demo.py` below runs after the Section 6 setup, before any TODO. Each pair is `(priority, node)`. These three priorities are different.

Code — examples/heap_demo.py

```
from collections import deque
import heapq

items = [(4, "A"), (1, "B"), (2, "C")]
fifo = deque(items)
print("FIFO first:", fifo.popleft()[1])

waiting = []
for item in items:
    heapq.heappush(waiting, item)
while waiting:
    priority, node = heapq.heappop(waiting)
    print(priority, node)
```

Run — Anaconda Prompt, repository root

```
python examples/heap_demo.py
```

Output

```
FIFO first: A
1 B
2 C
4 A
```

`popleft()[1]` reads A's name from the removed pair. The heap loop assigns each returned pair to `priority, node`, producing B, C, A. For k entries, heap insertion and removal take $O(\log k)$ time. The Assignment supplies these queue calls and an insertion counter that keeps arrival order for equal priorities. You implement cost improvement and priority calculation, not heap management.

2.2 Improve a node's cost and parent

Dijkstra's algorithm chooses the waiting node with the lowest known cost from the start. Write that cost as $g(u)$. Initially the start costs 0 and other costs are unknown (infinity). Moving from u to v proposes

$$g_{\text{candidate}} = g(u) + c(u, v),$$

where $c(u, v)$ is the edge cost. **Relaxation** replaces the stored cost and parent (the predecessor used to reach the node) only when the proposed cost is smaller. Equal costs keep the earlier parent.

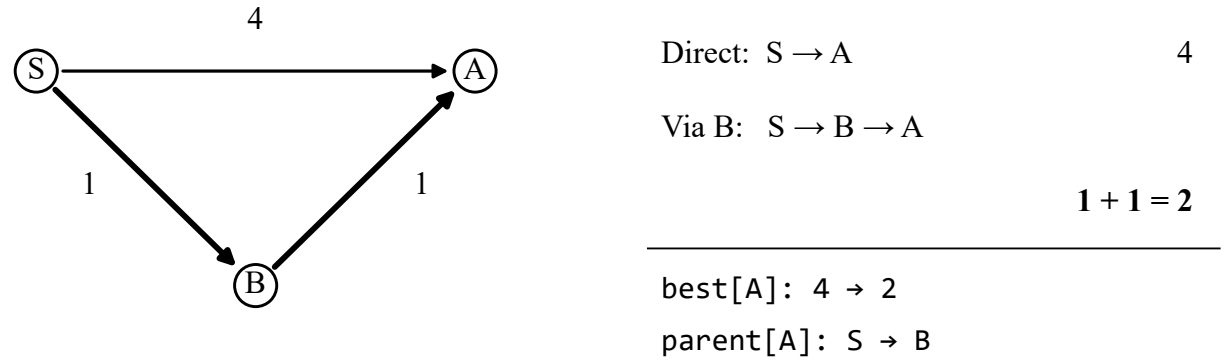


Figure 2: The direct edge S–A still costs 4. The cheaper route S–B–A costs 2, so the stored cost and parent of A are replaced.

An improvement leaves the old heap entry in place and pushes a new one. The older record is a **stale entry**. Skip it when its stored g exceeds `best[node]`, without recording or expanding it.

The table traces the complete graph in Figure 1. G's first cost of 9 is later improved to 5.

Removed node	Cost	Change
S	0	Discover A at 4, B at 1 and C at 2.
B	1	Improve A to 2 through B. Discover G at 9.
C	2	Discover D at 4.
A	2	Improve G to 5 through A.
A, old entry	4	Skip: A's current best cost is 2.
D	4	Route to G would cost 9. Keep 5.
G	5	Recover S–B–A–G and stop.

Stop when the goal is removed with its current best cost, not when it is first discovered. Following parents from G gives G–A–B–S. The supplied reconstruction helper reverses this to start-to-goal order.

Store g separately from priority because A* will use a different priority. If the heap empties without reaching the goal, return no path and infinite cost. If start equals goal, the one-node path costs 0.

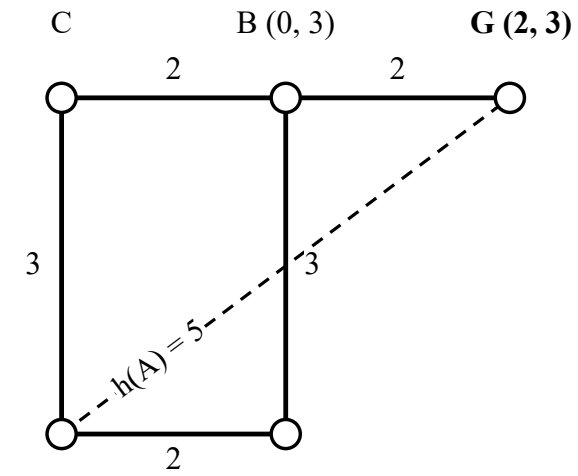
3 A*: use the goal to focus the search

A* considers both the cost already found and an estimate of the remaining cost. A **heuristic**, $h(u)$, supplies that estimate from node u to the goal. The priority is

$$\underbrace{f(u)}_{\text{priority}} = \underbrace{g(u)}_{\text{known cost from start}} + \underbrace{h(u)}_{\text{estimate to goal}} .$$

Figure 3 is a separate Reading-only map with (x, y) coordinates and metre costs. It is not Figure 1's graph or repository data. Its solid links permit movement in both directions.

After S: inspect A or B next?



Dijkstra
Cost from S

A: 2 ← **next**
B: 3

A*
Cost from S + estimate to G

A: 2 + 5 = 7
B: 3 + 2 = 5 ← **next**

Solid: either direction. Dashed: distance only, not a connection.
$h(A) = \sqrt{4^2 + 3^2} = 5$ (coordinate gaps: 4 across, 3 up)
$h(B) = 2$ (B and G have the same y-coordinate)

Both return S → B → G, cost 5 m. Their next choice differs.

Figure 3: A is cheaper to reach, but B has the smaller estimated total. The dashed measurement is not a connection or a new junction.

The straight distance of 5 m is exact, but A cannot move directly to G. Its shortest allowed continuation is 7 m, for example A–C–B–G (3 + 2 + 2). Thus $h(A) = 5$ estimates remaining *route* cost without first finding that route.

Dijkstra inspects S, A, B, C, G. A* inspects S, B, G and stops with A and C still waiting. Inspection order is not the route to execute. Equal priorities keep insertion order.

An **admissible heuristic** never exceeds the true minimum remaining cost. A route cannot be shorter than the direct distance. This distance heuristic is also **consistent**:

$$h(u) \le c(u, v) + h(v), \qquad h(\text{goal}) = 0.$$

Consistency means the estimate cannot drop by more than the edge cost. No proof is required here.

The loop still updates costs and parents and skips stale entries. Priority changes from g to $g + h$, but returned path cost remains g. Setting $h = 0$ gives Dijkstra.

The Assignment uses Section 1's graph with supplied `H` values in `cases.py`. Both searches return cost 5. Fewer inspections need not mean less execution time. Neither method searches outside its graph.

Choosing an estimate

Choose an estimate with the same units as cost, using the allowed movement to identify a valid lower bound.

Method	Calculation	When it fits
Euclidean distance	$\sqrt{(\Delta x)^2 + (\Delta y)^2}$	Distance cost. The robot example applies this to joint-angle differences.
Manhattan distance	$\lvert\Delta x\rvert + \lvert\Delta y\rvert$	Four-direction grid, no diagonal moves, unit distance cost per step.
Time lower bound	Direct distance / maximum speed	Time cost. Ignoring waits and detours gives a lower bound.
Zero estimate	$h = 0$	Nonnegative costs when no useful lower bound is available. A* becomes Dijkstra.

Do not use Manhattan distance unchanged when diagonal movement can be cheaper. A distance in metres is not a time estimate in seconds. Use the supplied heuristic for the Assignment, not these alternatives.

4 From node names to robot postures

A robot **configuration** is its set of joint values, representing a posture (also called a state here). In the supplied robot **roadmap**, q_1 rotates about the base axis and q_2 bends the shoulder. The other five joints are fixed. The six nodes in `cases.py` represent postures, not (x, y) locations. Figure 4 shows the returned route.

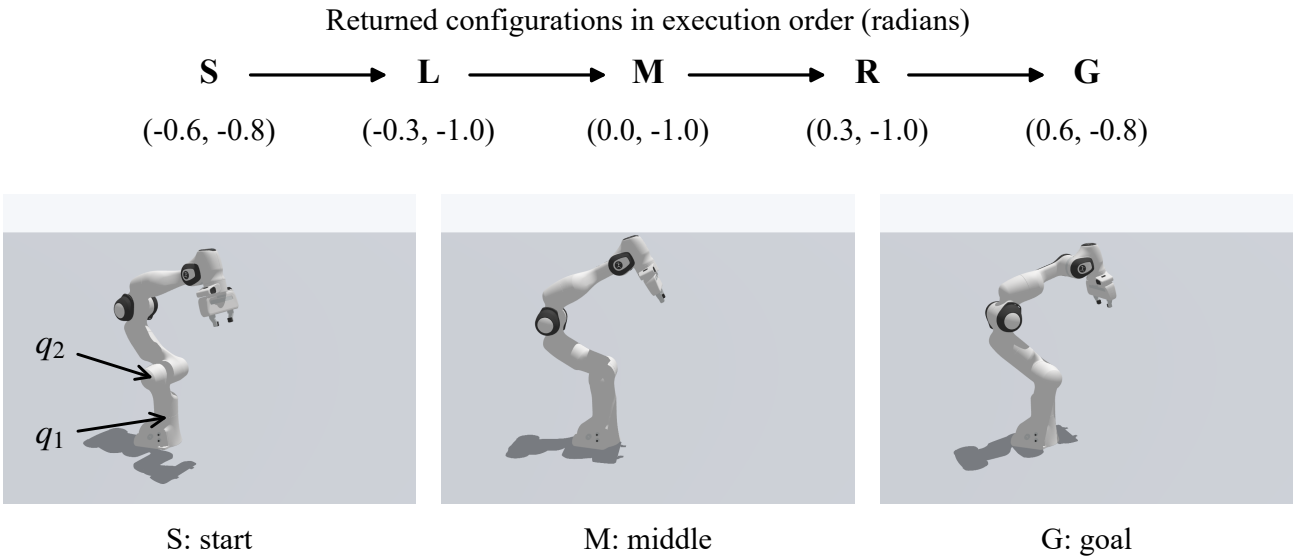


Figure 4: The first two joints are marked on S. Each node identifies a whole-robot posture. Top-row arrows show execution order, not a workspace path. Frames are from the supplied simulation.

S and G are Week 9 postures A and D. Supplied code converts selected node names into joint-angle rows. The GPU runner fills in five fixed joints and executes the array. RRT supplies new configurations to the same runner.

State validity checks a configuration. **Edge validity** checks the full movement between configurations. The checker uses narrower planning bounds than Week 9, with the same forbidden joint-angle region. This software restriction is not a physical box. S and G are valid, but their direct connection is rejected. The geometry is supplied.

Here edge cost is Euclidean joint-angle distance. The heuristic in `application.py` applies the same distance rule to the goal angles. S–L–M–R–G has length 1.321 rad. The objective is shorter joint-space length, not faster motion. GPU execution is a separate check, not real-robot safety certification.

5 RRT: find intermediate postures while searching

A **Rapidly-exploring Random Tree (RRT)** grows a tree from the start toward sampled configurations, finding intermediate postures instead of using only the supplied roadmap.

Dijkstra and A* can also generate neighboring states on demand. Missing intermediate postures therefore do not force RRT. Here the Assignment compares a supplied roadmap with a sampled tree.

5.1 Sample, extend and connect

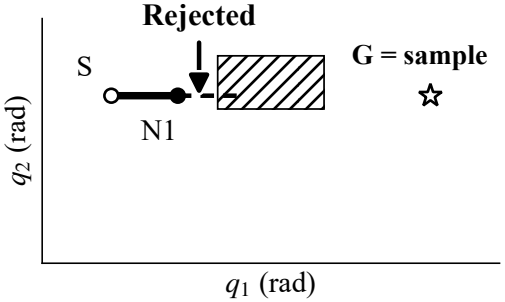
$G = (0.6, -0.8)$ rad stays fixed as the final goal. A **sample** guides one expansion and is named `target` in the code. Each time, choose G with probability 0.15. Otherwise, draw a posture uniformly within `problem.bounds`. Occasionally choosing the goal is called **goal bias**. G can be selected from the first expansion. The rule does not switch to goal-only sampling later.

Find the tree node nearest to the sample. Figure 5 starts with $S = (-0.6, -0.8)$ alone, so use S. Compute a nearby posture toward the sample (**steering**). If the connecting edge is valid, add this posture as a **new node** connected to the chosen tree node. Otherwise, keep the tree unchanged. These are planning calculations, not robot execution.

Why other samples are needed (step = 0.25 rad)

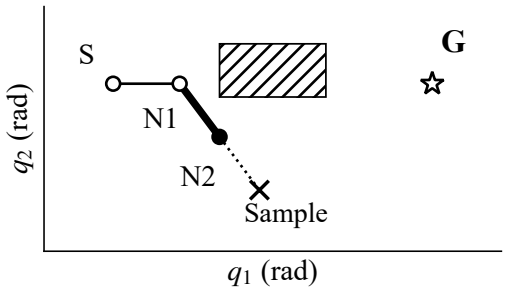
Hatched: forbidden joint angles. Filled: new. Dotted: toward sample. Dashed: rejected.

1. Two G samples: second step blocked



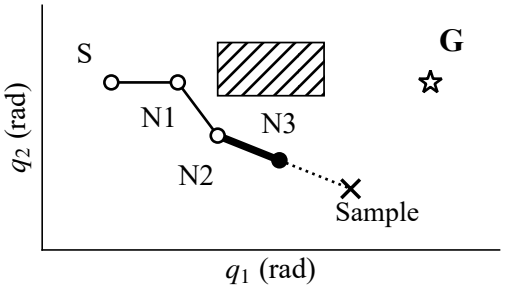
First step accepted. Next step rejected.
The rejected step adds no node.

2. Another sample changes direction



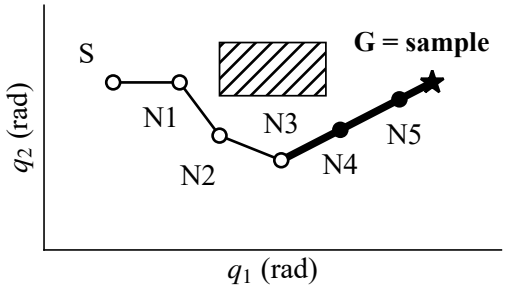
Other sample: (-0.05, -1.2).
Nearest: N1 (0.50 rad), not S (0.68 rad).

3. Keep growing below the box



Next sample: (0.3, -1.2).
A checked step adds N3 below the box.

4. Sample G again and connect



Two more G samples add N4 and N5.
Check the final 0.139 rad edge separately.

Figure 5: One possible sample sequence. The hatched region excludes joint-angle pairs, not workspace positions.

When a new node is within `step_size` of G, check the final connection. If valid, add G and recover the path. Figure 5 returns S–N1–N2–N3–N4–N5–G.

For $d = \|q_{\text{target}} - q_{\text{near}}\|_2 > 0$, the bounded step is

$$q_{\text{new}} = q_{\text{near}} + \min\left(1, \frac{\text{step_size}}{d}\right)(q_{\text{target}} - q_{\text{near}}).$$

Panel 2 uses N1 $= (-0.35, -0.8)$ and sample $(-0.05, -1.2)$. Their difference $(0.3, -0.4)$ has length 0.5, so the 0.25 step gives N2 $= (-0.2, -1.0)$. Closer targets are copied without overshooting. An identical target requires no division.

Each accepted node stores the index of the node it grew from (its parent). A parent need not be the most recently added node. The supplied `tree_path(nodes, parents, index)` follows the parents back to the start, then reverses that order to return the path. No additional A* search is needed. Basic RRT does not promise the shortest path.

5.2 Interpret a bounded run

A **random seed** initializes the random generator. Reusing the seed, inputs and implementation reproduces the samples, but does not guarantee success. A **planning budget** limits work. Here it counts edge checks, including rejected proposals and the final goal connection.

`FOUND` means a valid route reached the goal. `BUDGET_EXHAUSTED` means the available checks ended before finding one, not that a route is impossible. The supplied **RRT-Connect** grows two trees, one from each endpoint, and tries to connect them. You compare its result with your single-tree RRT, not implement it.

6 Week 10 Assignment

6.1 Purpose and files to edit

Implement cost-aware search and single-tree RRT, then execute one computed path with the supplied GPU runner. Complete the three TODOs below. Validation, reconstruction, RRT-Connect, robot execution and tests are supplied.

Use this table to locate your edits. In VS Code, replace each marked `raise NotImplementedError(...)` between BEGIN/END and save. Keep signatures and all code outside those markers unchanged.

TODO	File and function	Work to implement
1	`src/ap_week10/search.py` `weighted_search`	Cost improvement, predecessor update and priority calculation.
2	`src/ap_week10/rrt.py` `steer`	A bounded step toward a target.
3	`src/ap_week10/rrt.py` `single_tree_rrt`	Sample, choose a nearest state and grow one valid tree.

Edit only these two files.

6.2 Prepare the existing environment

Starter repository:

https://github.com/SSUMechE/applied-programming-2026-week10-starter

Use `Use this template` to create private `applied-programming-w10-YOUR-STUDENT-ID` and invite `SSUMechE`.

In Anaconda Prompt, replace the course folder, account and student ID below. Both `git remote -v` addresses must identify your private repository.

Run — Anaconda Prompt

```
cd /d "C:\your-course-folder"
git clone https://github.com/YOUR-GITHUB-ID/applied-programming-w10-YOUR-STUDENT-ID.git
cd applied-programming-w10-YOUR-STUDENT-ID
git remote -v
set PYTHONUTF8=1
conda activate applied-programming-cuda
python -m pip install "setuptools>=80.9.0" pytest==9.1.1
python -m pip install -e . --no-deps --no-build-isolation
python scripts/verify_environment.py
```

Reuse the earlier all-in-one CUDA/Genesis environment. Do not create a Week 10 environment or replace its GPU packages. If it is missing, follow the setup link in README. If verification fails, report the output instead of changing shared packages.

`ENVIRONMENT PASS` confirms imports, this checkout and a small CUDA calculation, not your TODOs. The supplied `examples/heap_demo.py` and `examples/inspect_problem.py` run before implementation. The latter displays the supplied inputs and `Direct edge allowed: False`.

6.3 TODO 1: improve costs and set priorities

Edit — src/ap_week10/search.py, weighted_search, TODO 1

Complete TODO 1 inside the provided neighbor loop. `graph[node]` contains ordered `(neighbor, cost)` pairs. At your writing region, `node` is the current node, `saved_g` is its cost from the start, and `neighbor, edge_cost` describe a permitted next edge. `heuristic(neighbor)` returns the remaining-cost estimate.

The supplied code initializes the queue, removes its smallest entry, skips stale entries, checks edges and returns the goal path. It also inserts improved entries and handles equal priorities. Your code uses `best` (known costs) and `parent` (predecessors).

1. Calculate `new_g` from `saved_g` and `edge_cost`. Compare it with `best.get(neighbor, float("inf"))`.
2. If the new cost is not smaller, use `continue` to skip this neighbor. Equal costs keep the existing predecessor.
3. For an improvement, store `new_g` in `best[neighbor]`, store `node` in `parent[neighbor]`, and assign `priority` as `new_g + heuristic(neighbor)`. The provided code then inserts this entry.

Keep all supplied code and inputs unchanged. Dijkstra passes a zero estimate, while A* passes a nonzero estimate to this same function. Only TODO 1 is needed for these commands.

Run — repository root, applied-programming-cuda

```
python examples/run_search.py
python -m pytest -q tests/test_search.py
```

Expected program output and test summary

```
Dijkstra: S -> B -> A -> G | cost=5.0
  expanded: S B C A D G
A*: S -> B -> A -> G | cost=5.0
  expanded: S B A G
Robot roadmap: S -> L -> M -> R -> G | cost=1.321 rad
4 passed
```

The first two costs use the teaching graph's cost units. The robot route uses joint-angle distance. Compare `path` with `expanded`: a processed node need not be on the returned path.

6.4 TODOs 2 and 3: grow a single RRT

Edit — src/ap_week10/rrt.py, steer, TODO 2

The supplied `origin` and `target` are finite `(2,)` NumPy arrays of joint angles in radians. Return a new `float64` array at most `step_size` toward the target, using Section 5's formula. Return an independent target copy when it is within one step, including zero distance. Do not change either input. Validation is supplied.

For example, `steer([-0.6, -0.8], [-0.3, -1.2], 0.25)` returns `[-0.45, -1.0]`.

Edit — src/ap_week10/rrt.py, single_tree_rrt, TODO 3

Complete only the marked expansion inside the supplied loop. `problem.start` and `problem.goal` are two-angle arrays. Set `low = problem.bounds[:, 0]` and `high = problem.bounds[:, 1]` for sampling. The supplied `nodes` starts with the start state, and `parents` starts with `None`. `parents[i]` records the predecessor index of `nodes[i]`.

The generator `rng`, timer `started` and `budget` are supplied. `budget.check(a, b)` counts a check and returns whether the whole edge between two configurations is valid. `budget.remaining` gives the checks left.

1. Set `target` to the goal when `rng.random() < 0.15`. Otherwise sample with `rng.uniform(low, high)`. Keep this fixed goal bias.
2. Choose the nearest index `i` using supplied `distance(nodes[i], target)`, the Euclidean joint-angle distance. Choose the first on a tie. Obtain `new_state` from `steer(nodes[i], target, step_size)`.
3. Call `budget.check(nodes[i], new_state)`. On rejection, continue. Otherwise append `new_state` to `nodes` and its parent index `i` to `parents`.
4. Return a path if the new state equals the goal. Otherwise check a final goal edge only when its distance is at most `step_size` and budget remains. If valid, append the goal and its parent index, then return the path.

For success, `tree_path(nodes, parents, last_index)` returns the start-to-goal array, where `last_index` identifies the goal. Return `finish(path, budget, len(nodes), started)` to package that array and its metrics. Start-equals-goal and budget exhaustion are already handled. Preserve the problem's arrays.

After TODOs 2 and 3, run the supplied comparison and tests. Both planners use seeds 7, 11 and 19, the same problem and step size, and at most 200 edge checks.

Run — repository root, applied-programming-cuda

```
python examples/run_sampling.py
python -m pytest -q tests/test_rrt.py
```

Expected seed-7 result lines and grouped test summary

```
RRT: FOUND | cost=1.504 rad | checks=28 | nodes=22
RRT-Connect: FOUND | cost=1.585 rad | checks=18 | nodes=11
4 passed
```

These are reference results, not exact-cost or node-count grading targets. Compare validity, path length, checks and elapsed time across seeds. Here RRT-Connect uses fewer checks but returns a longer path. No particular speed ratio is required.

6.5 Execute one completed path and submit

Execute one computed route on the GPU in the same environment. Use A* below, or `--planner rrt` after TODOs 2 and 3. CPU-only execution does not replace this check.

Run — repository root, applied-programming-cuda

```
python examples/run_robot_path.py --planner astar --capture
python -m pytest -q
```

Expect `Execution status: EXECUTED` and `8 passed`. Open `artifacts/astar/preview.html` (or `artifacts/rrt/preview.html`) to inspect this run's robot images. Tests alone do not verify GPU execution.

Review and push the two edited files. Do not commit environments, caches or credentials.

Run — repository root

```
git diff
git status --short
git add src/ap_week10/search.py src/ap_week10/rrt.py
git diff --staged
git commit -m "Complete Week 10 search algorithms"
git push origin main
git status --short
git rev-parse HEAD
git ls-remote origin refs/heads/main
```

Press `q` to leave a diff pager. Confirm an empty final status and matching full local/remote commit IDs. Complete all three conditions:

- Three TODOs complete, supplied checks pass, and one computed route executes on the GPU.
- Correct Week 10 private repository name and `SSUMechE` access. A correct invitation sent on time but still awaiting instructor acceptance is not a student omission.
- Submit the repository URL and pushed revision's full 40-character commit ID in LMS within one week of the lab, by the exact LMS date and time. GitHub upload alone is not LMS submission.

LMS example — replace the fictional URL and commit ID

```
Repository URL: https://github.com/student-example/applied-programming-w10-20260000
Commit ID: 0123456789abcdef0123456789abcdef01234567
```

Missing mandatory submission requirements at the deadline are subject to the announced zero-point rule, with the timely-pending invitation exception above. A failed algorithm test alone is not an automatic zero for the entire Assignment. No engineering note, wheel, ZIP, report or screenshot submission is required.